Test Case 1

University Scholarship Allocation System Using Python

1. Problem Statement

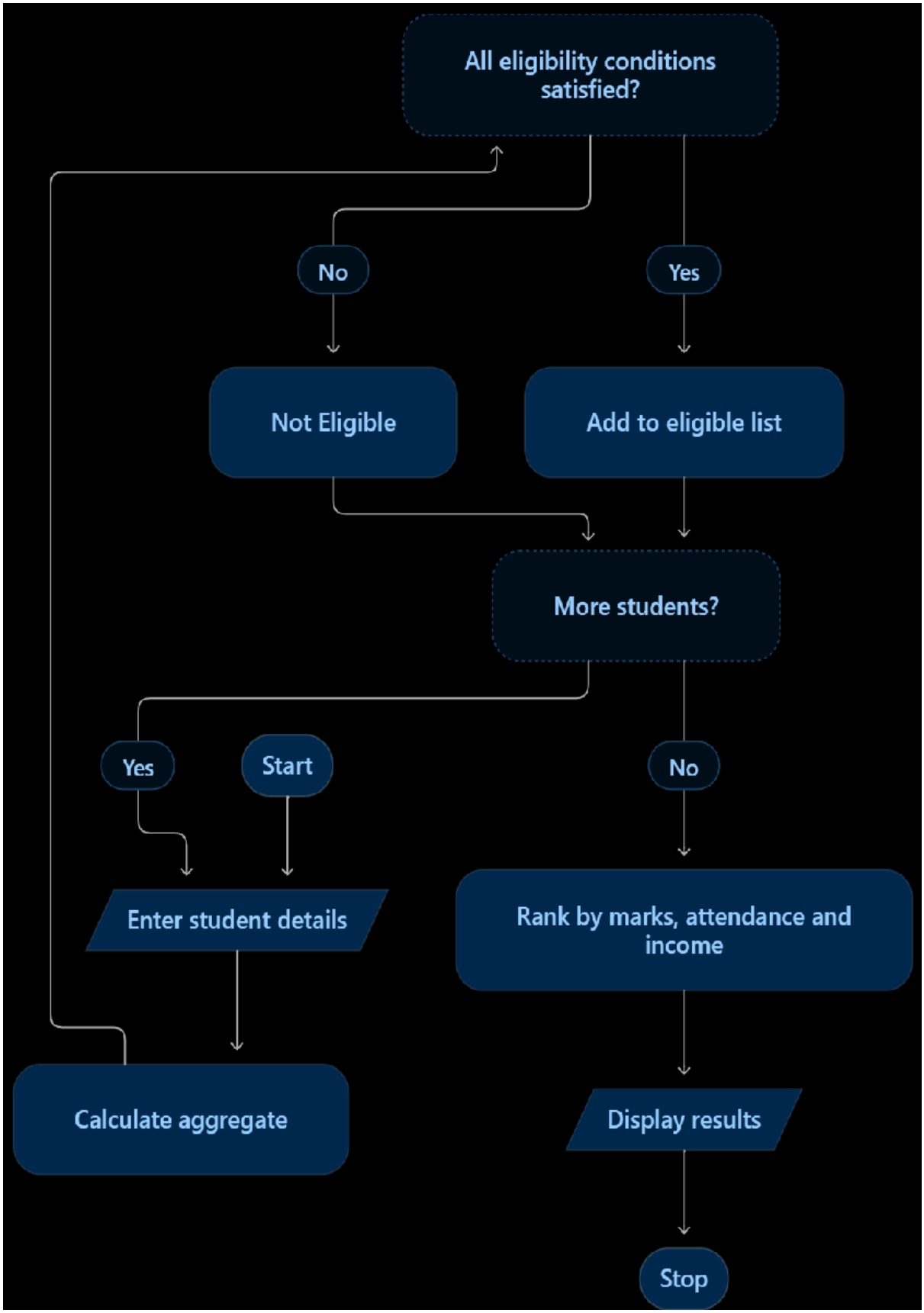
Develop a Python program to allocate university scholarships based on students' academic performance and financial background. The system evaluates students using aggregate marks, subject-wise minimum marks, attendance percentage, and annual family income.

The program identifies eligible students, ranks them based on their aggregate marks, and resolves ties using predefined rules such as higher attendance and lower family income.

2. Algorithm

1. Start the program.
2. Enter the number of students.
3. Set scholarship eligibility criteria:
    - Aggregate marks ≥ 75%.
    - Each subject mark ≥ 60%.
    - Attendance ≥ 75%.
    - Annual family income ≤ ₹2,50,000.
4. Enter each student's name, marks, attendance, and family income.
5. Calculate aggregate percentage.
6. Check all eligibility conditions.
7. Add eligible students to the scholarship list.
8. Rank eligible students using:
    - Higher aggregate percentage.
    - Higher attendance.
    - Lower family income.
9. Display the ranked scholarship list.
10. Stop the program.

3. Flowchart



4. Source Code

```
students = []
n = int(input("Enter number of students: "))
for i in range(n):
    name = input("Enter name: ")
    marks = list(map(int, input("Enter 5 subject marks: ").split()))
    att = float(input("Enter attendance: "))
    income = float(input("Enter family income: "))
    avg = sum(marks) / 5
    if avg >= 75 and min(marks) >= 60 and att >= 75 and income <= 250000:
        students.append((name, avg, att, income))
students.sort(key=lambda x: (-x[1], -x[2], x[3], x[0]))
print("\nEligible Students:")
for rank, s in enumerate(students, 1):
    print(rank, s)
if not students:
    print("No eligible students”)
```

5. Sample Input

```
Enter number of students: 3

Student 1
Enter name: Ravi
Enter 5 subject marks: 85 80 90 78 87
Enter attendance: 90
Enter annual family income: 200000

Student 2
Enter name: Kumar
Enter 5 subject marks: 80 85 75 90 80
Enter attendance: 85
Enter annual family income: 180000

Student 3
Enter name: Raju
Enter 5 subject marks: 55 80 75 90 85
Enter attendance: 95
Enter annual family income: 150000
```

6. Sample Output

```
--- Scholarship Eligible Students ---

Rank: 1
Name: Ravi
Aggregate: 84.0 %
Attendance: 90.0 %
Family Income: Rs. 200000.0

Rank: 2
Name: Kumar
Aggregate: 82.0 %
Attendance: 85.0 %
Family Income: Rs. 180000.0
```

Test Case 2

Progressive Electricity Billing System Using Python

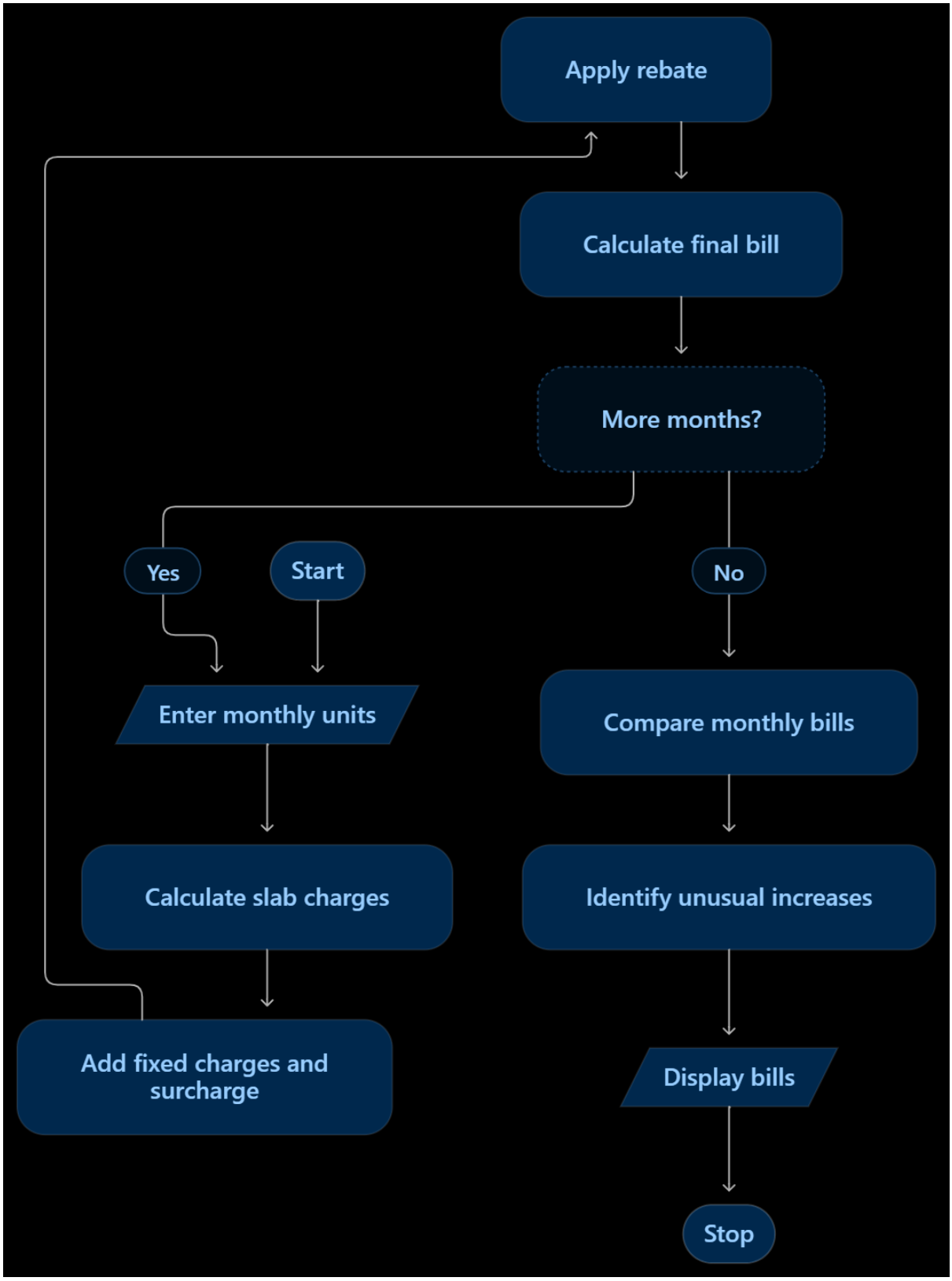
1. Problem Statement

Develop a Python program to calculate electricity bills based on consumption slabs, fixed charges, surcharges, and rebates. Compare bills across multiple months and identify unusual increases in electricity usage.

2. Algorithm

1. Start the program.
2. Enter the number of months.
3. Enter electricity units consumed each month.
4. Calculate charges using different slabs.
5. Add fixed charges and surcharges.
6. Apply rebates if applicable.
7. Calculate the final bill.
8. Compare monthly bills and identify unusual increases.
9. Display the bills.
10. Stop.

3. Flowchart



4.Source Code

```
bills = []
```

```
n = int(input("Enter number of months: "))
for i in range(n):
    units = int(input("Enter units consumed: "))
    if units <= 100:
        charge = units * 2
    elif units <= 200:
        charge = 100*2 + (units-100)*3
    else:
        charge = 100*2 + 100*3 + (units-200)*5
    fixed = 50
    surcharge = charge * 0.05
    rebate = 20 if units < 100 else 0
    total = charge + fixed + surcharge - rebate
    bills.append(total)
    print("Month", i+1, "Bill: Rs.", round(total, 2))
print("\nBill Comparison:")
for i in range(1, n):
    increase = bills[i] - bills[i-1]
    if increase > bills[i-1] * 0.20:
        print("Month", i+1, ": Unusual increase")
    else:
        print("Month", i+1, ": Normal")
```

5. Sample Input

```
Enter number of months: 3
Enter units consumed: 80
Enter units consumed: 150
Enter units consumed: 250
```

6. Sample Output

```
Month 1 Bill: Rs. 214.0
Month 2 Bill: Rs. 525.0
Month 3 Bill: Rs. 1050.0

Bill Comparison:
Month 2 : Unusual increase
Month 3 : Unusual increase
```

Test Case 3

Hospital Billing and Patient Classification Using Python

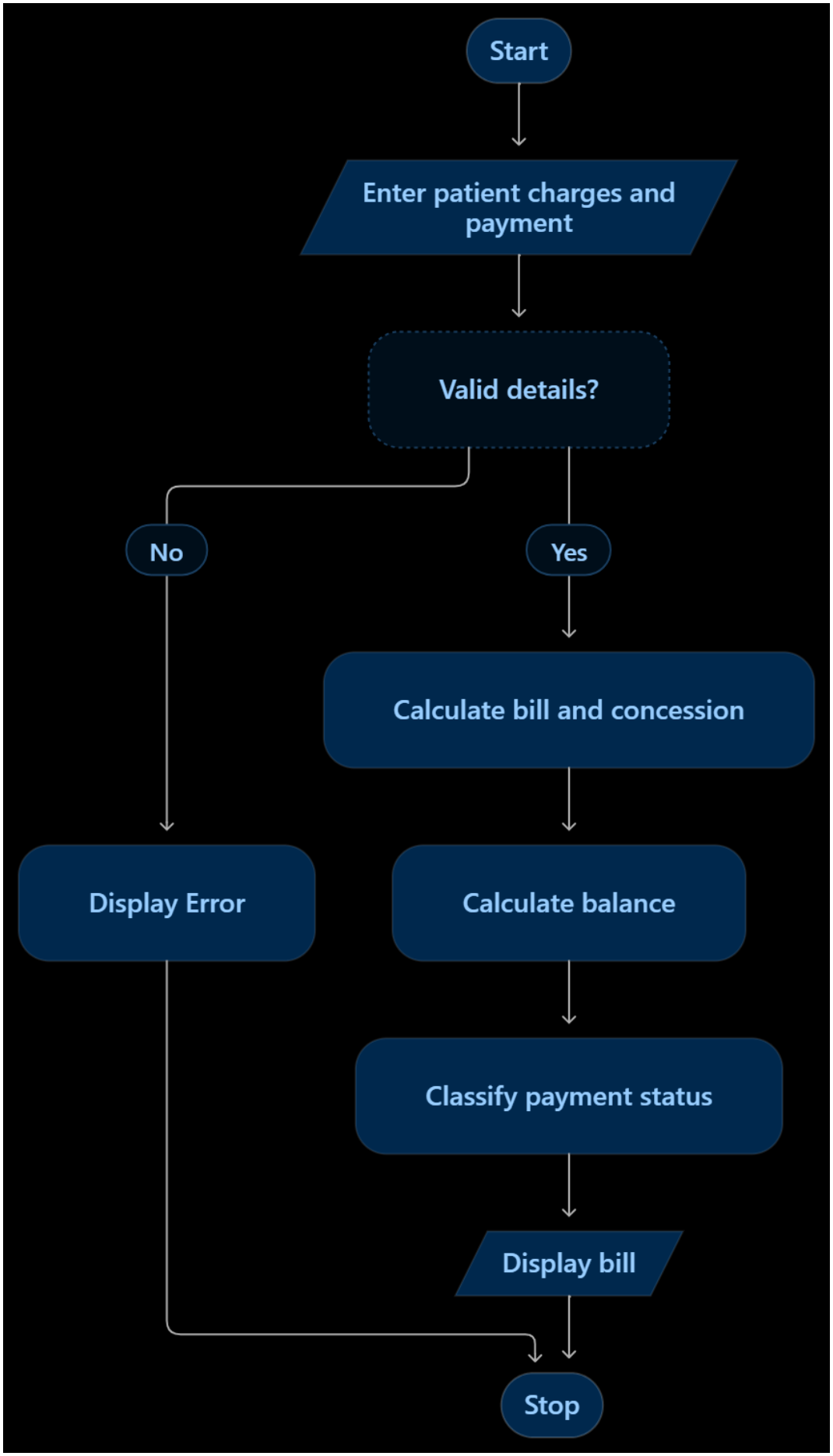
1. Problem Statement

Develop a Python program to calculate hospital bills using consultation, diagnostic, and treatment charges. Apply concessions based on patient conditions, validate billing details, and classify patients according to their payment status.

2. Algorithm

1. Start the program.
2. Enter patient name and charges.
3. Calculate total bill.
4. Apply concession if eligible.
5. Validate charges.
6. Calculate final bill.
7. Classify patient as Fully Paid, Partially Paid, or Unpaid.
8. Display bill details.
9. Stop.

3. Flowchart


Start
Enter patient charges and payment
Valid details?
No
Yes
Calculate bill and concession
Display Error
Calculate balance
Classify payment status
Display bill
Stop

4.Source Code

```
name = input("Enter patient name: ")
consult = float(input("Consultation charges: "))
diagnostic = float(input("Diagnostic charges: "))
treatment = float(input("Treatment charges: "))
payment = float(input("Enter payment amount: "))
if min(consult, diagnostic, treatment, payment) < 0:
    print("Invalid billing details")
else:
    total = consult + diagnostic + treatment
    discount = total * 0.10 if total > 5000 else 0
    final = total - discount
    if payment > final:
        print("Invalid payment amount")
    else:
        balance = final - payment
        if balance == 0:
            status = "Fully Paid"
        elif payment > 0:
            status = "Partially Paid"
        else:
            status = "Unpaid"
        print("\n--- Hospital Bill ---")
        print("Patient:", name)
        print("Total Bill:", total)
        print("Concession:", discount)
        print("Final Bill:", final)
        print("Payment:", payment)
```

```
print("Balance:", balance)
print("Classification:", status)
```

5. Sample Input

```
Enter patient name: Ravi
Consultation charges: 500
Diagnostic charges: 2000
Treatment charges: 4000
Enter payment amount: 3000
```

6. Sample Output

```
--- Hospital Bill ---
Patient: Ravi
Total Bill: 6500.0
Concession: 650.0
Final Bill: 5850.0
Payment: 3000.0
Balance: 2850.0
Classification: Partially Paid
```

Test Case 4

Employee Payroll and Deduction Analysis Using Python

1. Problem Statement

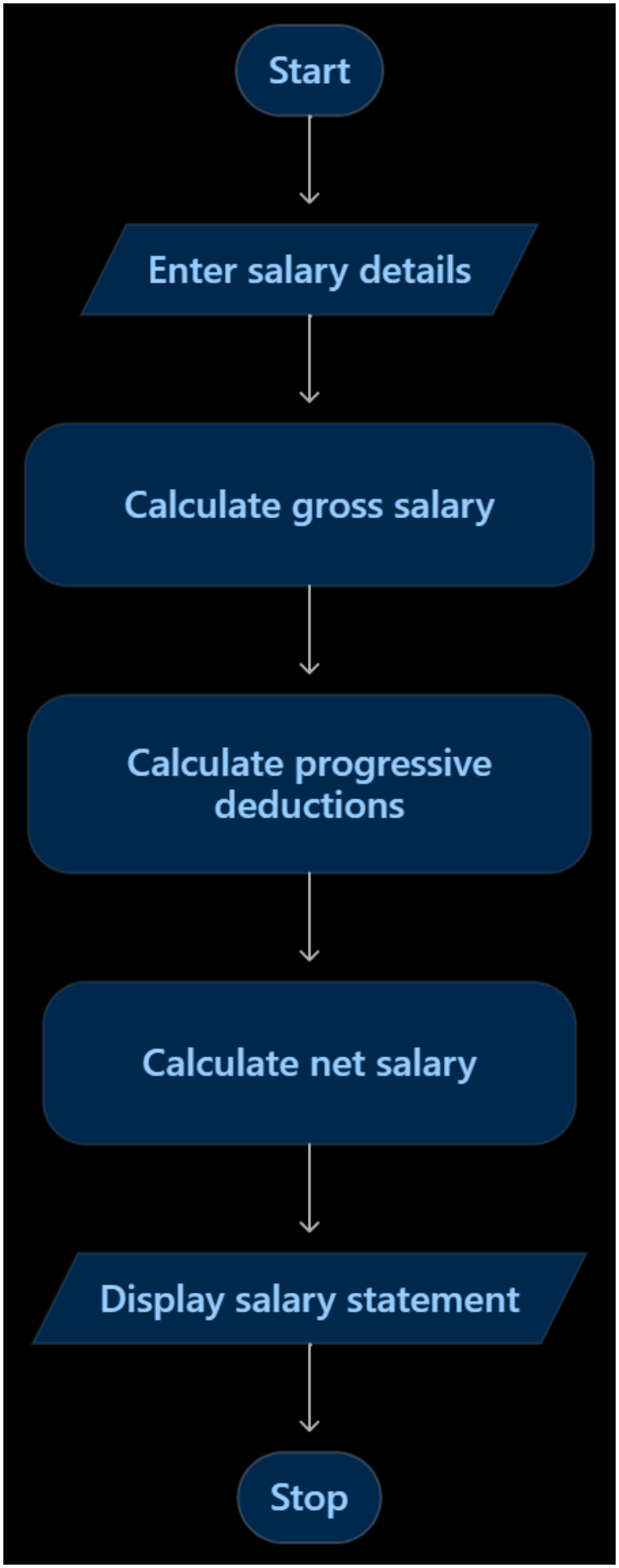
Develop a Python program to calculate employee gross salary using basic pay, allowances, overtime, and incentives. Apply progressive deductions and generate a formatted salary statement showing gross salary, deductions, and net salary.

2. Algorithm

1. Start the program.
2. Enter employee details and salary components.
3. Calculate gross salary.
4. Apply progressive deductions.
5. Calculate net salary.

6. Display the salary statement.
7. Stop.

3. Flowchart


Start
Enter salary details
Calculate gross salary
Calculate progressive deductions
Calculate net salary
Display salary statement
Stop

4.Source Code

```
name = input("Enter employee name: ")
basic = float(input("Enter basic pay: "))
allowance = float(input("Enter allowances: "))
overtime = float(input("Enter overtime pay: "))
incentive = float(input("Enter incentives: "))
gross = basic + allowance + overtime + incentive
if gross <= 20000:
    deduction = gross * 0.05
elif gross <= 40000:
    deduction = 20000*0.05 + (gross-20000)*0.10
else:
    deduction = 20000*0.05 + 20000*0.10 + (gross-40000)*0.15
net = gross - deduction
print("\n--- Salary Statement ---")
print("Employee:", name)
print("Gross Salary: Rs.", gross)
print("Deduction: Rs.", deduction)
print("Net Salary: Rs.", net)
```

5. Sample Input

```
Enter employee name: Ravi
Enter basic pay: 25000
Enter allowances: 5000
Enter overtime pay: 2000
Enter incentives: 3000
```

6. Sample Output

```
--- Salary Statement ---
Employee: Ravi
Gross Salary: Rs. 35000.0
Deduction: Rs. 2500.0
Net Salary: Rs. 32500.0
```

Test Case 5

Multi-Plan Telecommunications Cost Optimization Using Python

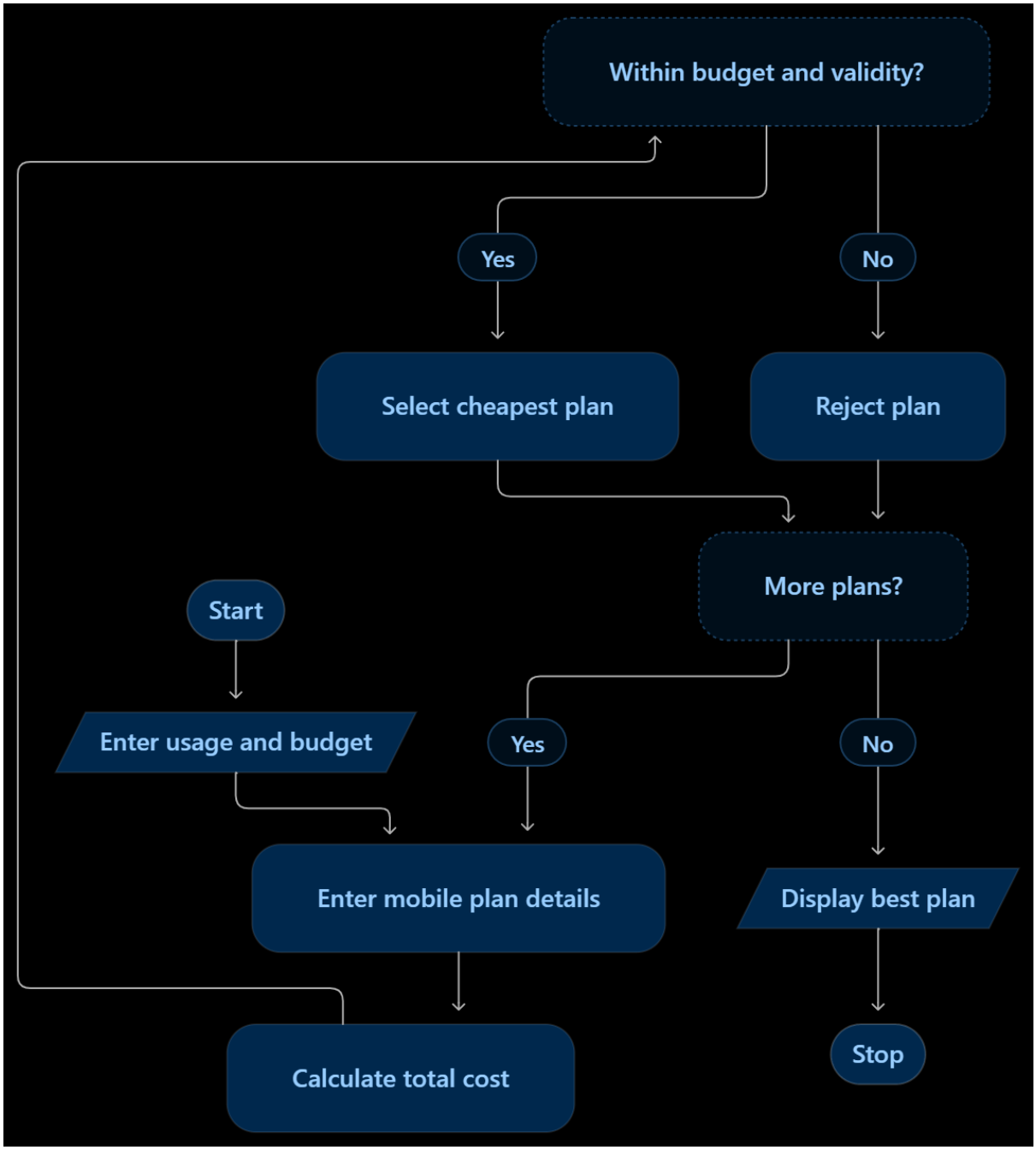
1. Problem Statement

Develop a Python program to compare different mobile plans based on data usage, validity, and budget. Calculate excess usage charges and identify the most economical eligible mobile plan.

2. Algorithm

1. Start the program.
2. Enter data usage, validity, and budget.
3. Enter mobile plan details.
4. Calculate excess charges.
5. Calculate total plan cost.
6. Check budget and validity conditions.
7. Select the cheapest eligible plan.
8. Display the result.
9. Stop.

3. Flowchart



4.Source Code

```
usage = float(input("Enter data usage (GB): "))
days = int(input("Enter required validity: "))
budget = float(input("Enter budget: "))

plans = []
n = int(input("Enter number of plans: "))
```

```
for i in range(n):
    name = input("Plan name: ")
    price = float(input("Plan price: "))
    data = float(input("Data limit (GB): "))
    validity = int(input("Validity (days): "))
    extra = float(input("Extra charge per GB: "))

    excess = max(0, usage - data)
    total = price + excess * extra

    if total <= budget and validity >= days:
        plans.append((name, total))

if plans:
    best = min(plans, key=lambda x: x[1])
    print("\nBest Plan:", best[0])
    print("Total Cost: Rs.", best[1])
else:
    print("No eligible plan found")
```

5. Sample Input

```
Enter data usage (GB): 20
Enter required validity: 28
Enter budget: 500
Enter number of plans: 3

Plan name: Plan A
Plan price: 299
Data limit (GB): 15
Validity (days): 28
Extra charge per GB: 10

Plan name: Plan B
Plan price: 399
Data limit (GB): 25
Validity (days): 28
Extra charge per GB: 10

Plan name: Plan C
Plan price: 199
Data limit (GB): 10
Validity (days): 14
```

```
Extra charge per GB: 15
```

6. Sample Output

```
Best Plan: Plan B
Total Cost: Rs. 399.0
```

Test Case 6

Railway Fare and Reservation Calculator Using Python

1. Problem Statement

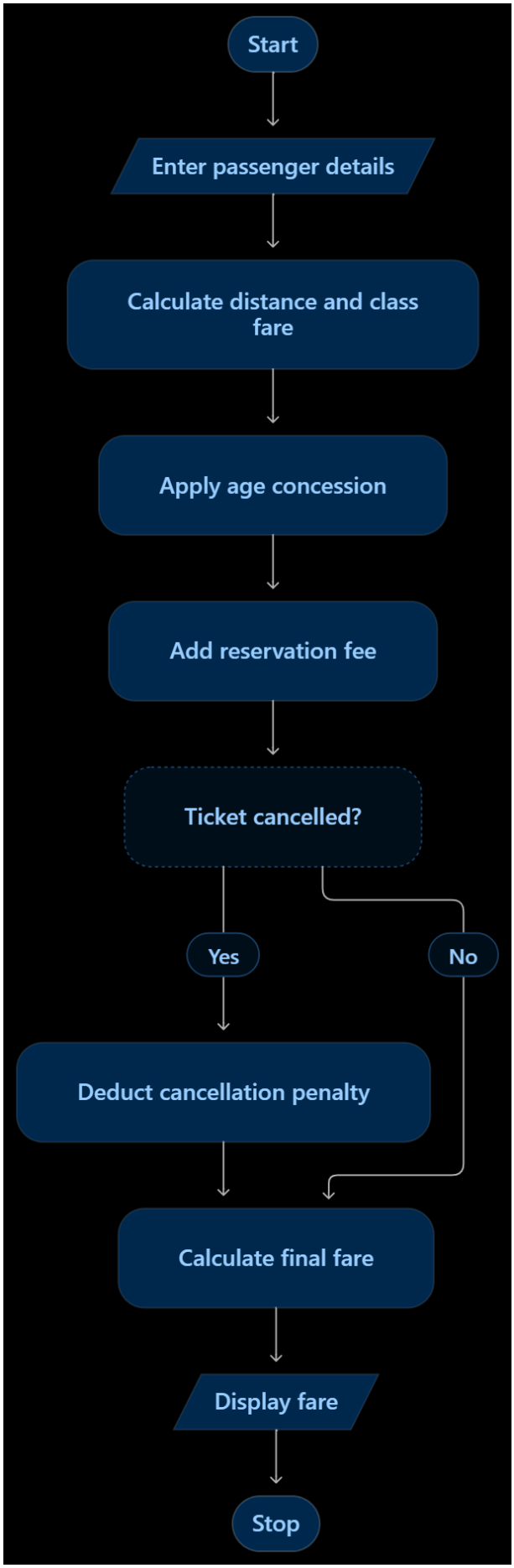
Develop a Python program to calculate railway fares for multiple passengers based on distance slabs, travel classes, age-based concessions, reservation fees, and cancellation penalties.

2. Algorithm

1. Start the program.
2. Enter the number of passengers.
3. Enter passenger age, distance, and travel class.

4. Calculate fare using distance slabs.
5. Apply age-based concessions.
6. Add reservation fees.
7. Calculate cancellation penalty if applicable.
8. Display the final fare.
9. Stop.

3. Flowchart


Start
Enter passenger details
Calculate distance and class fare
Apply age concession
Add reservation fee
Ticket cancelled?
Yes
No
Deduct cancellation penalty
Calculate final fare
Display fare
Stop

4.Source Code

```
n = int(input("Enter number of passengers: "))

for i in range(n):
    print("\nPassenger", i+1)
    name = input("Enter name: ")
    age = int(input("Enter age: "))
    distance = float(input("Enter distance (km): "))
    cls = input("Enter class (Sleeper/AC): ").lower()

    rate = 2 if cls == "sleeper" else 5
    fare = distance * rate

    if age < 5:
        fare = 0
    elif age >= 60:
        fare *= 0.5

    reservation = 50
    total = fare + reservation

    cancel = input("Cancel ticket? (yes/no): ").lower()

    if cancel == "yes":
        penalty = total * 0.20
        total -= penalty

    print("\n--- Railway Ticket ---")
```

```
    print("Passenger:", name)
    print("Fare: Rs.", fare)
    print("Reservation: Rs.", reservation)
    print("Final Fare: Rs.", round(total, 2))
```

5. Sample Input

```
Enter number of passengers: 2

Passenger 1
Enter name: Ravi
Enter age: 65
Enter distance (km): 500
Enter class (Sleeper/AC): sleeper
Cancel ticket? (yes/no): no

Passenger 2
Enter name: Kumar
Enter age: 30
Enter distance (km): 500
Enter class (Sleeper/AC): AC
Cancel ticket? (yes/no): yes
```

6. Sample Output

```
--- Railway Ticket ---
Passenger: Ravi
Fare: Rs. 500.0
Reservation: Rs. 50
Final Fare: Rs. 550.0

--- Railway Ticket ---
Passenger: Kumar
Fare: Rs. 2500.0
Reservation: Rs. 50
Final Fare: Rs. 2040.0
```

Testing Case 7

Retail Billing with Multiple Discount Rules Using Python

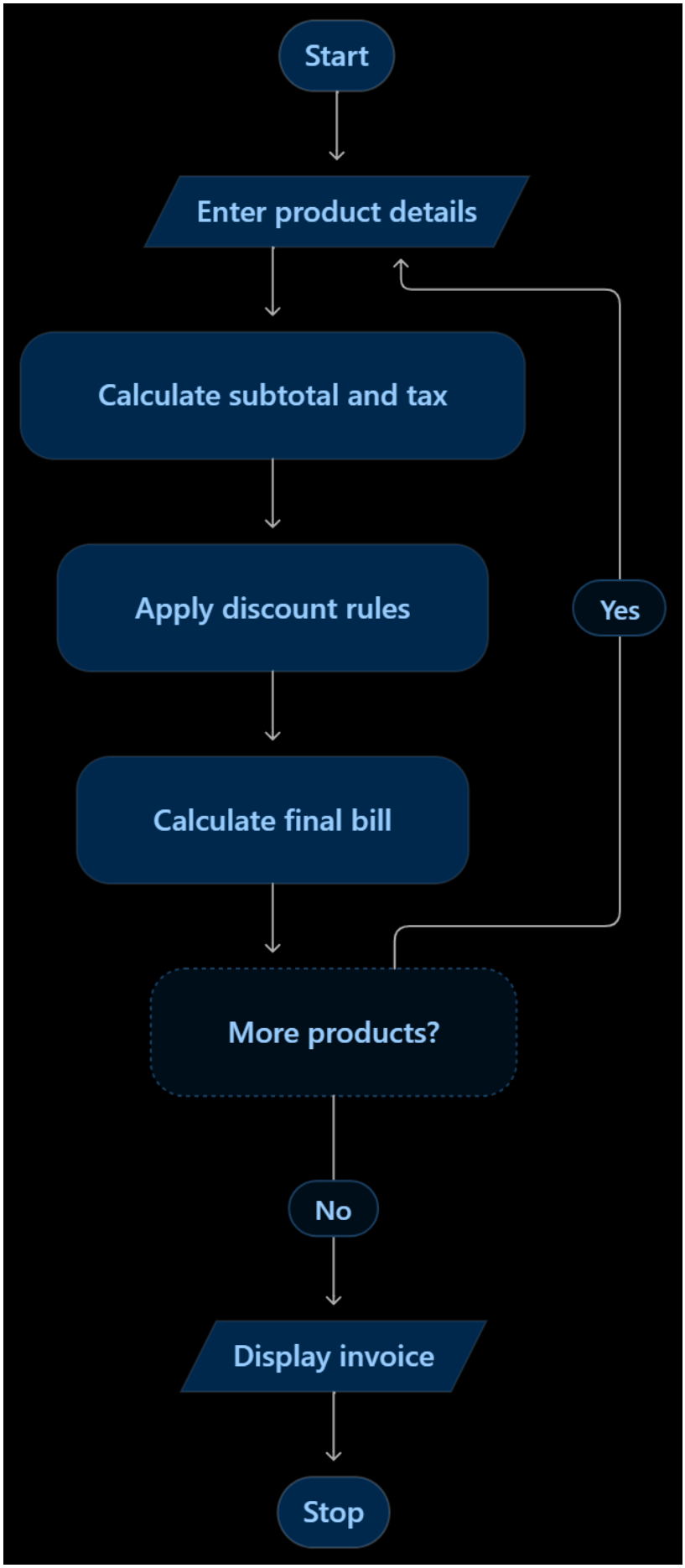
1. Problem Statement

Develop a Python program to process multiple products, calculate quantities and category-specific taxes, apply threshold discounts and maximum concession limits, and generate an itemized invoice.

2. Algorithm

1. Start the program.
2. Enter the number of products.
3. Enter product name, category, price, and quantity.
4. Calculate subtotal and category tax.
5. Apply threshold discount.
6. Apply maximum concession limit.
7. Calculate final bill.
8. Display itemized invoice.
9. Stop.

3. Flowchart


Start
Enter product details
Calculate subtotal and tax
Apply discount rules
Calculate final bill
More products?
Yes
No
Display invoice
Stop

4.Source Code

```
n = int(input("Enter number of products: "))
total = 0

for i in range(n):
    name = input("Product name: ")
    category = input("Category (food/electronics): ").lower()
    price = float(input("Price: "))
    qty = int(input("Quantity: "))

    amount = price * qty

    tax_rate = 5 if category == "food" else 18
    tax = amount * tax_rate / 100

    print(name, "Amount:", amount, "Tax:", tax)
    total += amount + tax

discount = total * 0.10 if total >= 5000 else 0
discount = min(discount, 500)

final = total - discount

print("\n--- Retail Invoice ---")
print("Subtotal with tax:", round(total, 2))
print("Discount:", round(discount, 2))
print("Final Bill: Rs.", round(final, 2))
```

5.Sample Input

```
Enter number of products: 2

Product name: Rice
Category: food
Price: 1000
Quantity: 2

Product name: Mobile
Category: electronics
Price: 10000
Quantity: 1
```

6.Sample Output

```
Rice Amount: 2000.0 Tax: 100.0
Mobile Amount: 10000.0 Tax: 1800.0

--- Retail Invoice ---
Subtotal with tax: 13900.0
Discount: 500.0
Final Bill: Rs. 13400.0
```

Test Case 8

Manufacturing Cost and Profitability Analysis Using Python

1. Problem Statement

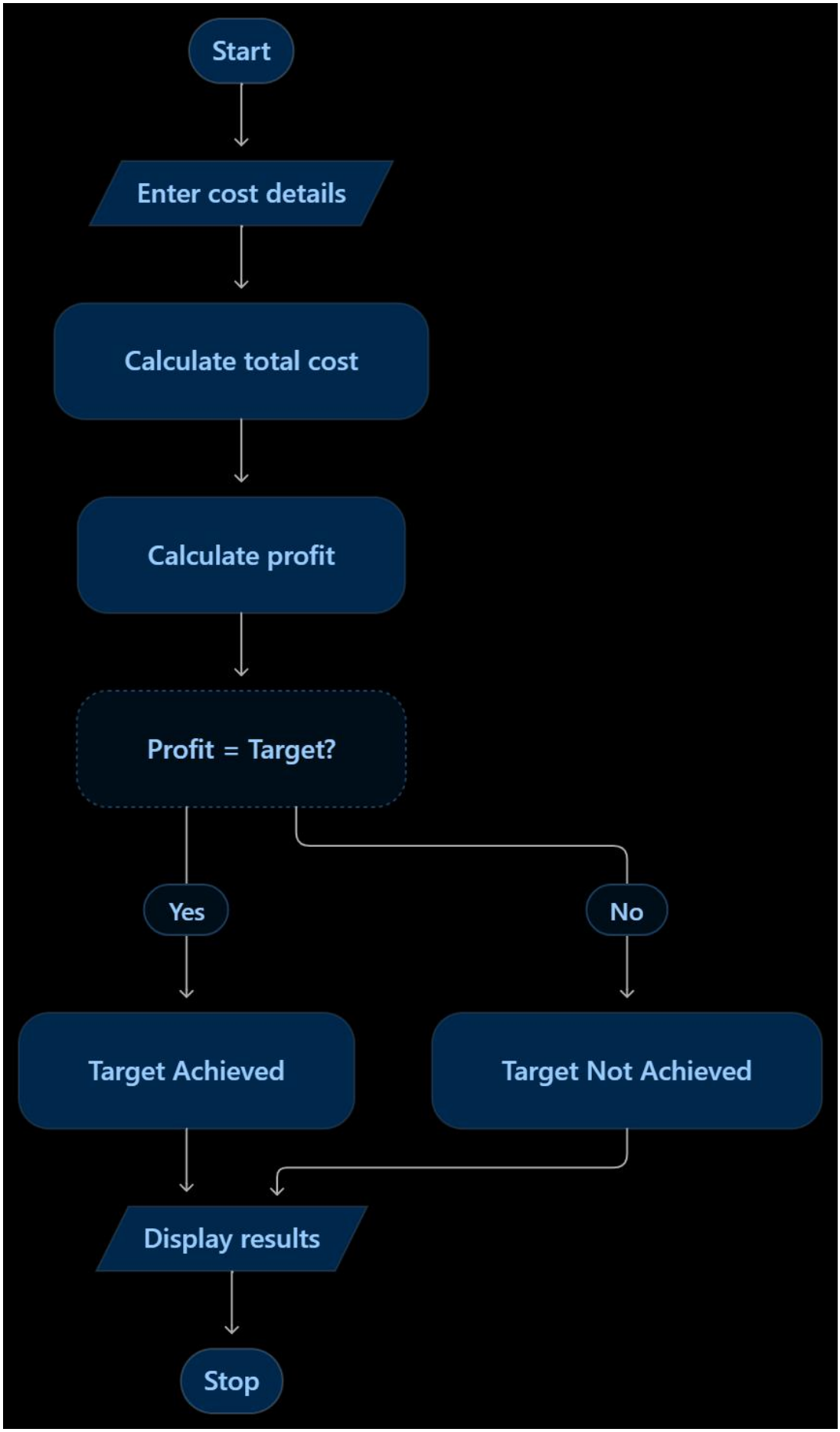
Develop a Python program to calculate manufacturing costs using material costs, labour charges, wastage, and machine costs. Calculate selling price and determine whether production meets the specified profit target.

2. Algorithm

1. Start the program.
2. Enter material, labour, and machine costs.
3. Enter wastage percentage and selling price.
4. Calculate total production cost.

5. Calculate total profit.
6. Compare profit with target.
7. Display the result.
8. Stop.

3. Flowchart


Start
Enter cost details
Calculate total cost
Calculate profit
Profit = Target?
Yes
No
Target Achieved
Target Not Achieved
Display results
Stop

4.Source Code

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour charges: "))
machine = float(input("Enter machine cost: "))
wastage = float(input("Enter wastage percentage: "))
selling = float(input("Enter selling price: "))
target = float(input("Enter profit target: "))

waste_cost = material * wastage / 100
total_cost = material + labour + machine + waste_cost
profit = selling - total_cost

print("\n--- Manufacturing Report ---")
print("Total Cost:", total_cost)
print("Selling Price:", selling)
print("Profit:", profit)

if profit >= target:
    print("Profit Target Achieved")
else:
    print("Profit Target Not Achieved")
```

5. Sample Input

```
Enter material cost: 10000
Enter labour charges: 3000
Enter machine cost: 2000
Enter wastage percentage: 10
Enter selling price: 18000
Enter profit target: 2000
```

6. Sample Output

```
--- Manufacturing Report ---
Total Cost: 16000.0
Selling Price: 18000.0
Profit: 2000.0
Profit Target Achieved
```

Test Case 9

Multi-Currency Travel Budget Analysis Using Python

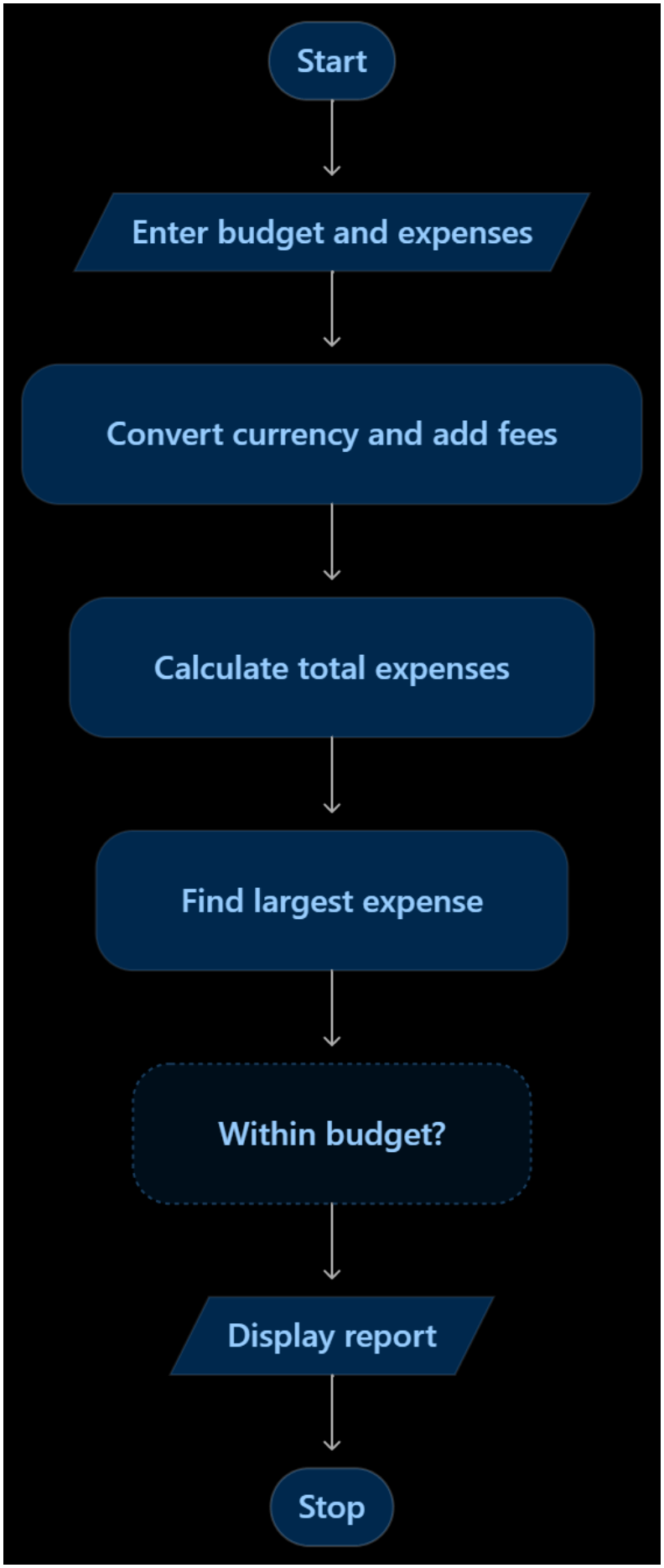
1. Problem Statement

Develop a Python program to calculate travel expenses in different currencies using supplied exchange rates. Include conversion fees, compare total spending against the budget, and identify the largest expense category.

2. Algorithm

1. Start the program.
2. Enter travel budget in INR.
3. Enter expense categories and amounts.
4. Enter currency and exchange rate.
5. Convert expenses into INR.
6. Add currency conversion fees.
7. Calculate total expenses.
8. Identify the largest expense category.
9. Compare expenses with budget.
10. Display the report and stop.

3. Flowchart


Start
Enter budget and expenses
Convert currency and add fees
Calculate total expenses
Find largest expense
Within budget?
Display report
Stop

4.Source Code

```
budget = float(input("Enter budget in INR: "))
n = int(input("Enter number of expenses: "))
expenses = {}

for i in range(n):
    category = input("Enter category: ")
    amount = float(input("Enter expense amount: "))
    rate = float(input("Enter exchange rate to INR: "))
    fee = float(input("Enter conversion fee percentage: "))

    total = amount * rate
    total += total * fee / 100

    expenses[category] = total
    print(category, "Cost: Rs.", round(total, 2))

total_cost = sum(expenses.values())
largest = max(expenses, key=expenses.get)

print("\n--- Travel Budget Report ---")
print("Total Expenses: Rs.", round(total_cost, 2))
print("Largest Expense:", largest)

if total_cost <= budget:
    print("Within Budget")
else:
    print("Over Budget by Rs.", round(total_cost-budget, 2))
```

5. Sample Input

```
Enter budget in INR: 100000
Enter number of expenses: 3

Enter category: Hotel
Enter expense amount: 300
Enter exchange rate to INR: 85
Enter conversion fee percentage: 2

Enter category: Food
Enter expense amount: 100
Enter exchange rate to INR: 85
Enter conversion fee percentage: 2

Enter category: Transport
Enter expense amount: 200
Enter exchange rate to INR: 85
Enter conversion fee percentage: 2
```

6. Sample Output

```
Hotel Cost: Rs. 26010.0
Food Cost: Rs. 8670.0
Transport Cost: Rs. 17340.0

--- Travel Budget Report ---
Total Expenses: Rs. 52020.0
Largest Expense: Hotel
Within Budget
```

Test Case 10

Campus Event Financial Planning Using Python

1. Problem Statement

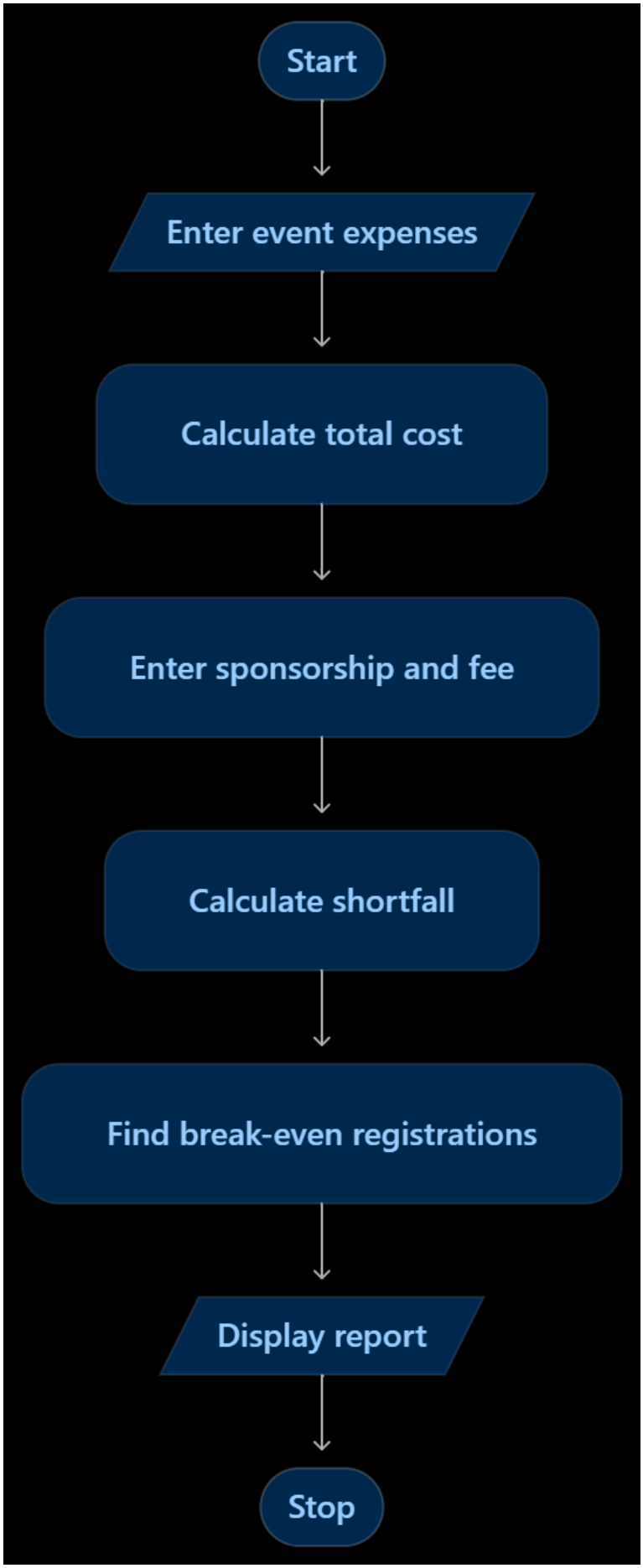
Develop a Python program to calculate campus event expenses, including venue, equipment, catering, transport, and certificates. Calculate participant contributions, sponsorship shortfalls, and minimum registrations required to break even.

2. Algorithm

1. Start the program.
2. Enter all event expenses.

3. Calculate total cost.
4. Enter sponsorship and registration fee.
5. Calculate sponsorship shortfall.
6. Calculate participant contribution.
7. Find minimum registrations required.
8. Display the financial report.
9. Stop.

3. Flowchart


Start
Enter event expenses
Calculate total cost
Enter sponsorship and fee
Calculate shortfall
Find break-even registrations
Display report
Stop

4.Source Code

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost: "))
transport = float(input("Enter transport cost: "))
certificates = float(input("Enter certificate cost: "))

sponsorship = float(input("Enter sponsorship amount: "))
fee = float(input("Enter registration fee per person: "))
participants = int(input("Enter expected participants: "))

total = venue + equipment + catering + transport + certificates

shortfall = max(0, total - sponsorship)
contribution = shortfall / participants if participants > 0 else 0

if fee > 0:
    minimum = max(0, int((total - sponsorship + fee - 0.01) // fee))
else:
    minimum = 0 if sponsorship >= total else -1

print("\n--- Event Financial Report ---")
print("Total Cost: Rs.", total)
print("Sponsorship Shortfall: Rs.", shortfall)
print("Contribution per Participant: Rs.", round(contribution, 2))

if minimum >= 0:
    print("Minimum Registrations:", minimum)
```

```
else:
    print("Break-even not possible without registration fee")
```

5. Sample Input

```
Enter venue cost: 10000
Enter equipment cost: 5000
Enter catering cost: 20000
Enter transport cost: 5000
Enter certificate cost: 2000
Enter sponsorship amount: 12000
Enter registration fee per person: 300
Enter expected participants: 100
```

6. Sample Output

```
--- Event Financial Report ---
Total Cost: Rs. 42000.0
Sponsorship Shortfall: Rs. 30000.0
Contribution per Participant: Rs. 300.0
Minimum Registrations: 100
```